



Tutorial

Introducción a *Catalogs* y su configuración en Kedro

El **Data Catalog** es uno de los componentes clave de Kedro.

Actúa como un registro centralizado de todas las fuentes y destinos de datos disponibles en el proyecto. En esencia, funciona como un inventario que asigna nombres a los conjuntos de datos (*datasets*), especificando tanto su ubicación física como la forma en que deben ser cargados o almacenados (Kedro, s.f.).

Este catálogo proporciona una capa de abstracción entre el código del *pipeline* y los datos, lo que aporta múltiples ventajas:

- ▶ **Separación de responsabilidades:** tu código de procesamiento no necesita conocer los detalles de cómo leer o escribir datos.
- ▶ **Configuración centralizada:** todas las fuentes de datos se definen en un solo lugar.
- ▶ **Flexibilidad:** puedes cambiar fácilmente las ubicaciones de los datos sin cambiar tu código.
- ▶ **Reproducibilidad:** las especificaciones de datos están versionadas junto con tu código.

Estructura básica del *Data Catalog*

En Kedro, el ***Data Catalog*** se configura principalmente mediante archivos YAML, específicamente a través de un archivo llamado `catalog.yml` (Kedro, s.f.).

YAML es un lenguaje de serialización, es decir, un formato utilizado para representar estructuras de datos de manera que puedan ser almacenadas o transmitidas fácilmente (por ejemplo, a través de archivos de texto o paquetes de red) y que, al mismo tiempo, resulte legible para los humanos.

Su característica más distintiva es el uso de sangrías (espacios) para definir la estructura y el anidamiento. A diferencia de formatos como JSON o XML, que emplean llaves, corchetes o etiquetas, YAML representa la jerarquía de los datos mediante la alineación espacial.

```
# Ejemplo de mapeo simple
nombre: Juan Perez
edad: 30
ciudad: "Ciudad de México" # Las son opcionales aquí,
pero válidas activo: true
```

Además, YAML representa mapeos (equivalentes a diccionarios en Python, *hashes* en Perl u objetos en JavaScript) usando pares llave-valor. La sintaxis es simple: **clave: valor**. También requiere incluir un espacio después de los dos puntos.

```
# Ejemplo de mapeo anidado
usuario:
  nombre: Ana García
  direccion:
    calle: Calle Con Nombre 123
    ciudad: Springfield
  roles:
    - editor
    - publicador
  departamentos: [ventas, contabilidad]
```

Las cadenas de caracteres utilizadas como llaves o valores, generalmente no requieren comillas, salvo cuando incluyen caracteres especiales, espacios al inicio o al final, o podrían confundirse con otros tipos de datos, como números o valores booleanos.

Por otro lado, los mapeos pueden anidarse mediante sangría, la cual debe realizarse exclusivamente con espacios, nunca con tabuladores).

Las secuencias (equivalentes a listas en Python o arreglos en JavaScript) se representan con un guion (-) seguido de un espacio al inicio de cada elemento, colocados con sangría bajo la clave principal o, alternativamente, utilizando corchetes. Por ejemplo:
Los comentarios se indican con #

Nota: Puedes consultar la especificación completa en <https://yaml.org/>

```
Nombre_del_dataset:  
type: tipo_del_dataset  
filepath: ruta/al/archivo  
# Opciones adicionales específicas del tipo de dataset  
load_args:  
# Argumentos para cargar datos  
save_args:  
# Argumentos para guardar datos
```

En Kedro, el catálogo se define utilizando estas reglas. Dentro de estos archivos, cada entrada sigue una estructura similar y, en el catálogo, tiene al menos:

- ▶ Un **nombre** (la clave en el archivo YAML).
- ▶ Un **tipo** (que especifica la clase de conjunto de datos a utilizar).
- ▶ Opciones para especificar dónde y cómo se almacenan los datos (como *filepath*).

Configuración básica del *Data Catalog*

Antes de comenzar, se requiere crear un proyecto de Kedro, para ello, se recomienda hacerlo dentro de un entorno virtual, ubicado en una carpeta específica, tal como se explicó en el Tema 1.

Una vez creado y activado el ambiente virtual procedemos a crear el nuevo proyecto con el comando `kedro new` (Kedro, n.d.) Este comando activa un asistente interactivo que te guiará paso a paso en la configuración inicial del proyecto:

1. Te pedirá un nombre para el proyecto.
2. Te preguntará qué herramientas se incluirán en el proyecto; las más comunes son Lint, Test, Log, Docs, Data Folder y Kedro-Viz. Es importante no seleccionar PySpark, a menos que realmente se vaya a utilizar.
3. Te dará la opción de incluir un *pipeline* de ejemplo; sin embargo, para un proyecto de producción, la respuesta siempre deber ser "no".

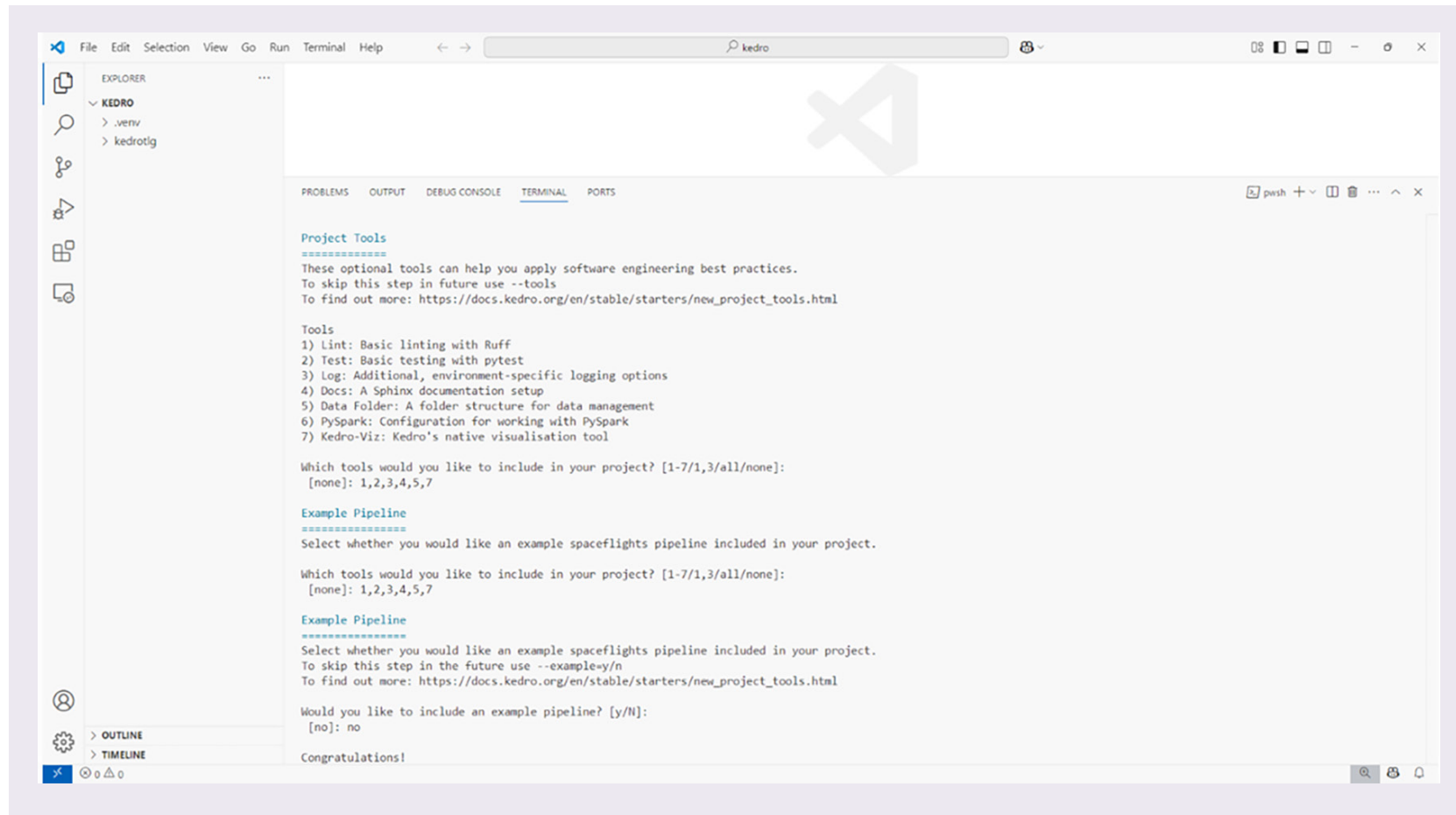


Figura 1. Página para crear nuevo proyecto.

En el ejemplo de la imagen, se configuró un proyecto llamado KedroTLG, se seleccionaron las herramientas Lint, Test, Log, Docs, Data Folder y Kedro-Viz y no se incluyó un *pipeline* de ejemplo.

Kedro creó la estructura de carpetas mostrada en la siguiente imagen:

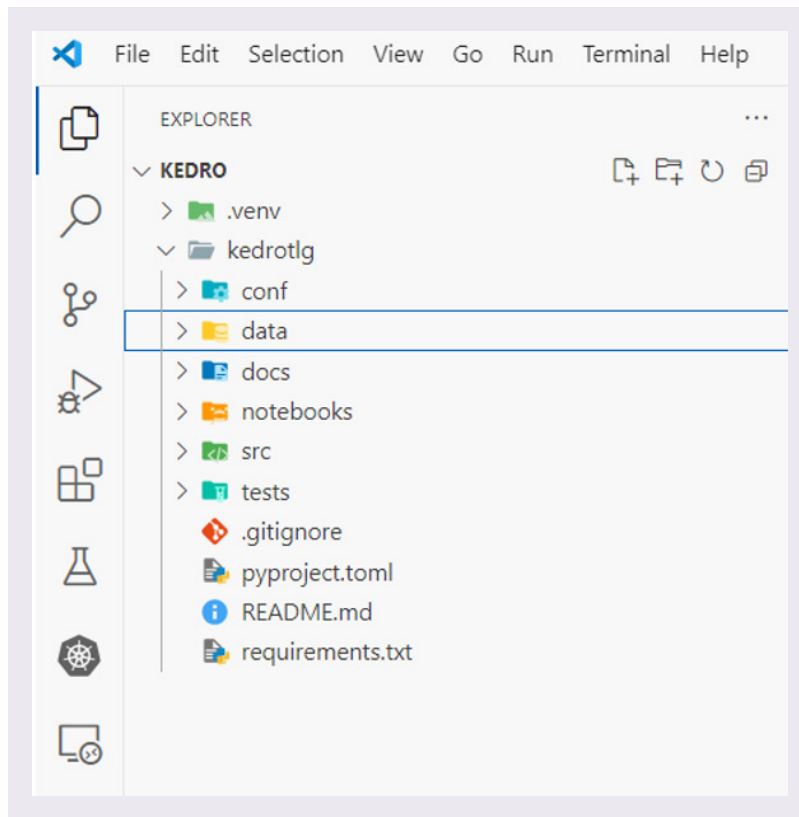


Figura 2. Estructura de carpetas creada por Kedro.

En el archivo `requirements.txt` habrá que agregar las bibliotecas que requeriremos para el proyecto, por ejemplo, Pandas o NumPy.

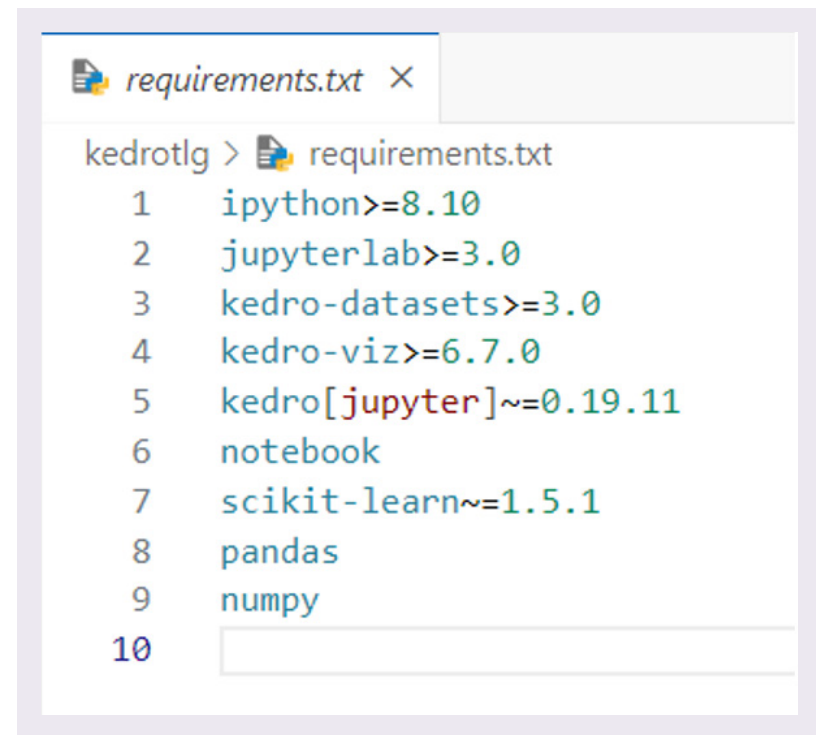
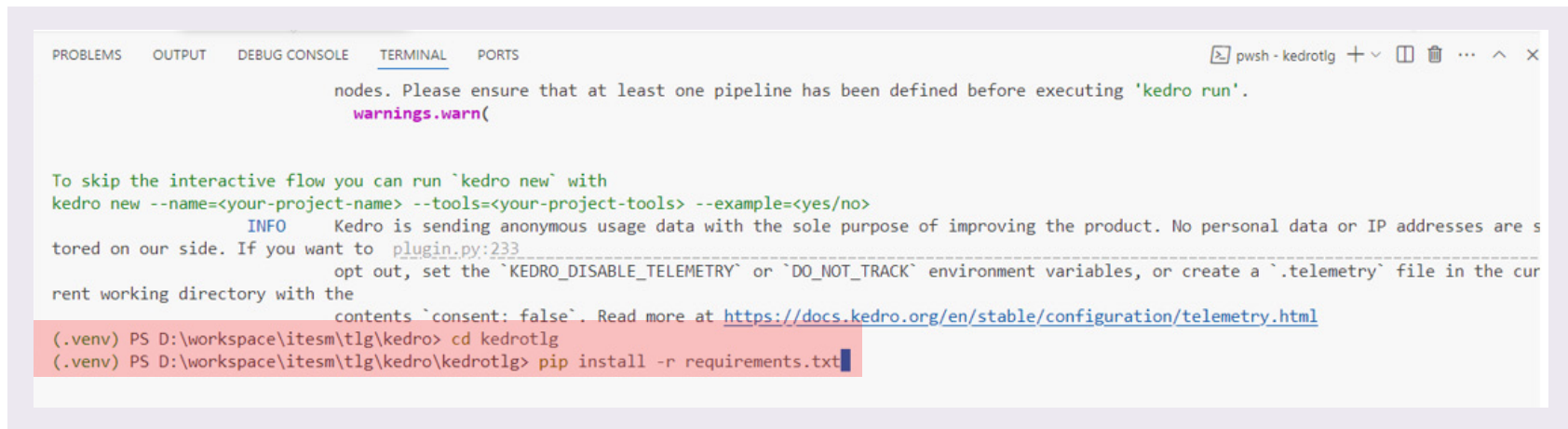


Figura 3. Archivo `requirements.txt`.

Si utilizamos **pip**, es importante mantener actualizado el archivo `requirements.txt`. Para las herramientas que utilizan `pyproject.toml`, se deben incluir las bibliotecas en el nodo **dependencies**.

Una vez actualizado el archivo, procedemos a instalar las dependencias. Para esto, en una terminal entramos al directorio del proyecto con el comando `cd <nombre del directorio>` y ejecutamos el comando `pip install -r requirements.txt`.

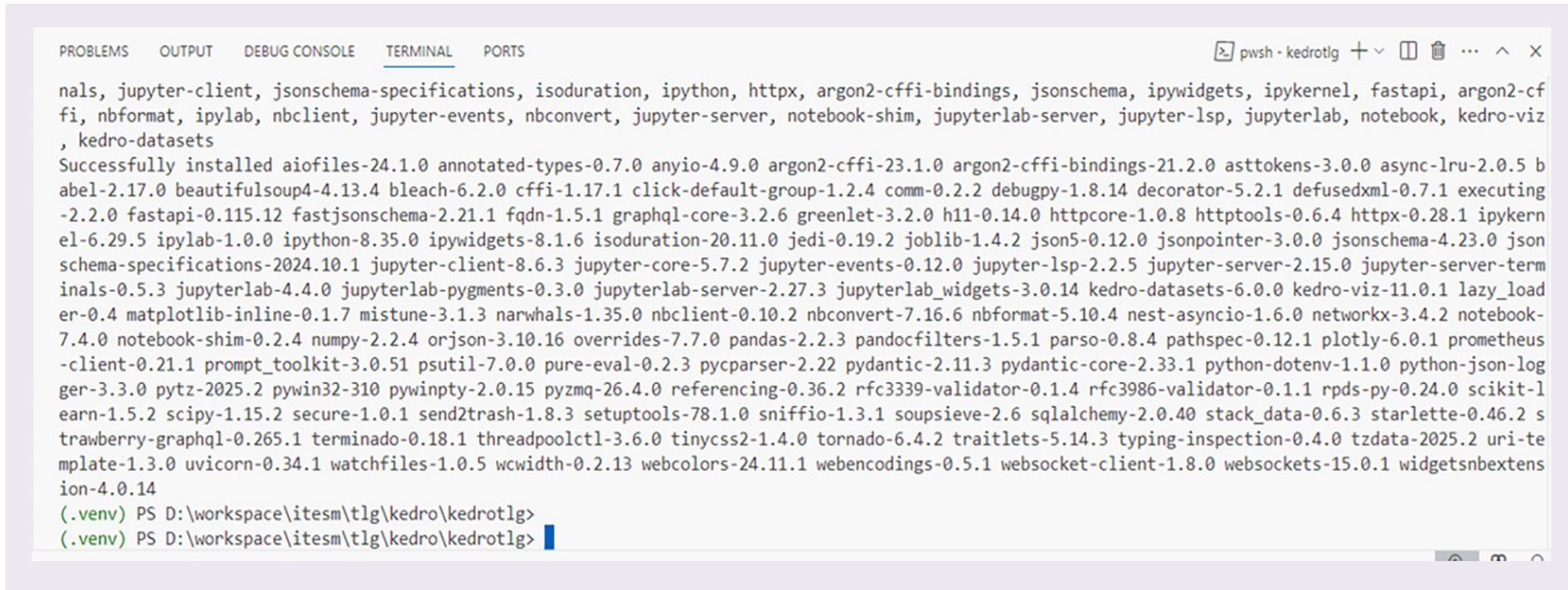
A screenshot of a terminal window with a light blue header bar containing tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal shows the output of a Kedro command, including a warning about pipeline definition and information about anonymous usage data. The last two lines of the terminal are highlighted in red: `(.env) PS D:\workspace\itesm\tlg\kedro> cd kedro_tlg` and `(.env) PS D:\workspace\itesm\tlg\kedro\kedro_tlg> pip install -r requirements.txt`.

```
nodes. Please ensure that at least one pipeline has been defined before executing 'kedro run'.
warnings.warn(

To skip the interactive flow you can run `kedro new` with
kedro new --name=<your-project-name> --tools=<your-project-tools> --example=<yes/no>
INFO      Kedro is sending anonymous usage data with the sole purpose of improving the product. No personal data or IP addresses are s
tored on our side. If you want to plugin.py:233
opt out, set the `KEDRO_DISABLE_TELEMETRY` or `DO_NOT_TRACK` environment variables, or create a `.telemetry` file in the cur
rent working directory with the
contents `consent: false`. Read more at https://docs.kedro.org/en/stable/configuration/telemetry.html
(.env) PS D:\workspace\itesm\tlg\kedro> cd kedro_tlg
(.env) PS D:\workspace\itesm\tlg\kedro\kedro_tlg> pip install -r requirements.txt
```

Figura 4. Pantalla de Terminal.

Toma en cuenta que esta instalación puede tardar muchos minutos.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
nals, jupyter-client, jsonschema-specifications, isoduration, ipython, httpx, argon2-cffi-bindings, jsonschema, ipywidgets, ipykernel, fastapi, argon2-cffi, nbformat, ipylab, nbclient, jupyter-events, nbconvert, jupyter-server, notebook-shim, jupyterlab-server, jupyter-lsp, jupyterlab, notebook, kedro-viz, kedro-datasets
Successfully installed aiofiles-24.1.0 annotated-types-0.7.0 anyio-4.9.0 argon2-cffi-23.1.0 argon2-cffi-bindings-21.2.0 asttokens-3.0.0 async-lru-2.0.5 babel-2.17.0 beautifulsoup4-4.13.4 bleach-6.2.0 cffi-1.17.1 click-default-group-1.2.4 comm-0.2.2 debugpy-1.8.14 decorator-5.2.1 defusedxml-0.7.1 executing-2.2.0 fastapi-0.115.12 fastjsonschema-2.21.1 fqdn-1.5.1 graphql-core-3.2.6 greenlet-3.2.0 h11-0.14.0 httpcore-1.0.8 httptools-0.6.4 httpx-0.28.1 ipykernel-6.29.5 ipylab-1.0.0 ipython-8.35.0 ipywidgets-8.1.6 isoduration-20.11.0 jedi-0.19.2 joblib-1.4.2 json5-0.12.0 jsonpointer-3.0.0 jsonschema-4.23.0 jsonschema-specifications-2024.10.1 jupyter-client-8.6.3 jupyter-core-5.7.2 jupyter-events-0.12.0 jupyter-lsp-2.2.5 jupyter-server-2.15.0 jupyter-server-terminals-0.5.3 jupyterlab-4.4.0 jupyterlab-pygments-0.3.0 jupyterlab-server-2.27.3 jupyterlab-widgets-3.0.14 kedro-datasets-6.0.0 kedro-viz-11.0.1 lazy_loader-0.4 matplotlib-inline-0.1.7 mistune-3.1.3 narwhals-1.35.0 nbclient-0.10.2 nbconvert-7.16.6 nbformat-5.10.4 nest-asyncio-1.6.0 networkx-3.4.2 notebook-7.4.0 notebook-shim-0.2.4 numpy-2.2.4 orjson-3.10.16 overrides-7.7.0 pandas-2.2.3 pandocfilters-1.5.1 parso-0.8.4 pathspec-0.12.1 plotly-6.0.1 prometheus-client-0.21.1 prompt-toolkit-3.0.51 psutil-7.0.0 pure-eval-0.2.3 pycparser-2.22 pydantic-2.11.3 pydantic-core-2.33.1 python-dotenv-1.1.0 python-json-logger-3.3.0 pytz-2025.2 pywin32-310 pywinpty-2.0.15 pyzmq-26.4.0 referencing-0.36.2 rfc3339-validator-0.1.4 rfc3986-validator-0.1.1 rpds-py-0.24.0 scikit-learn-1.5.2 scipy-1.15.2 secure-1.0.1 send2trash-1.8.3 setuptools-78.1.0 sniffio-1.3.1 soupsieve-2.6 sqlalchemy-2.0.40 stack-data-0.6.3 starlette-0.46.2 strawberry-graphql-0.265.1 terminado-0.18.1 threadpoolctl-3.6.0 tinycss2-1.4.0 tornado-6.4.2 traitlets-5.14.3 typing-inspection-0.4.0 tzdata-2025.2 uri-template-1.3.0 uvicorn-0.34.1 watchfiles-1.0.5 wcwidth-0.2.13 webcolors-24.11.1 webencodings-0.5.1 websocket-client-1.8.0 websockets-15.0.1 widgetsnbextension-4.0.14
(.venv) PS D:\workspace\itesm\tlg\kedro\kedro_tlg>
(.venv) PS D:\workspace\itesm\tlg\kedro\kedro_tlg>
```

Figura 5. Pantalla de Terminal ejecutando.

Para este proyecto utilizaremos el conjunto de datos disponible en el archivo **datos.zip**, descargado desde Kaggle (<https://www.kaggle.com/datasets/mrmorj/dataset-of-songs-in-spotify>). Una vez descargado, descomprime el archivo y copia su contenido en la carpeta **data/01_raw**.

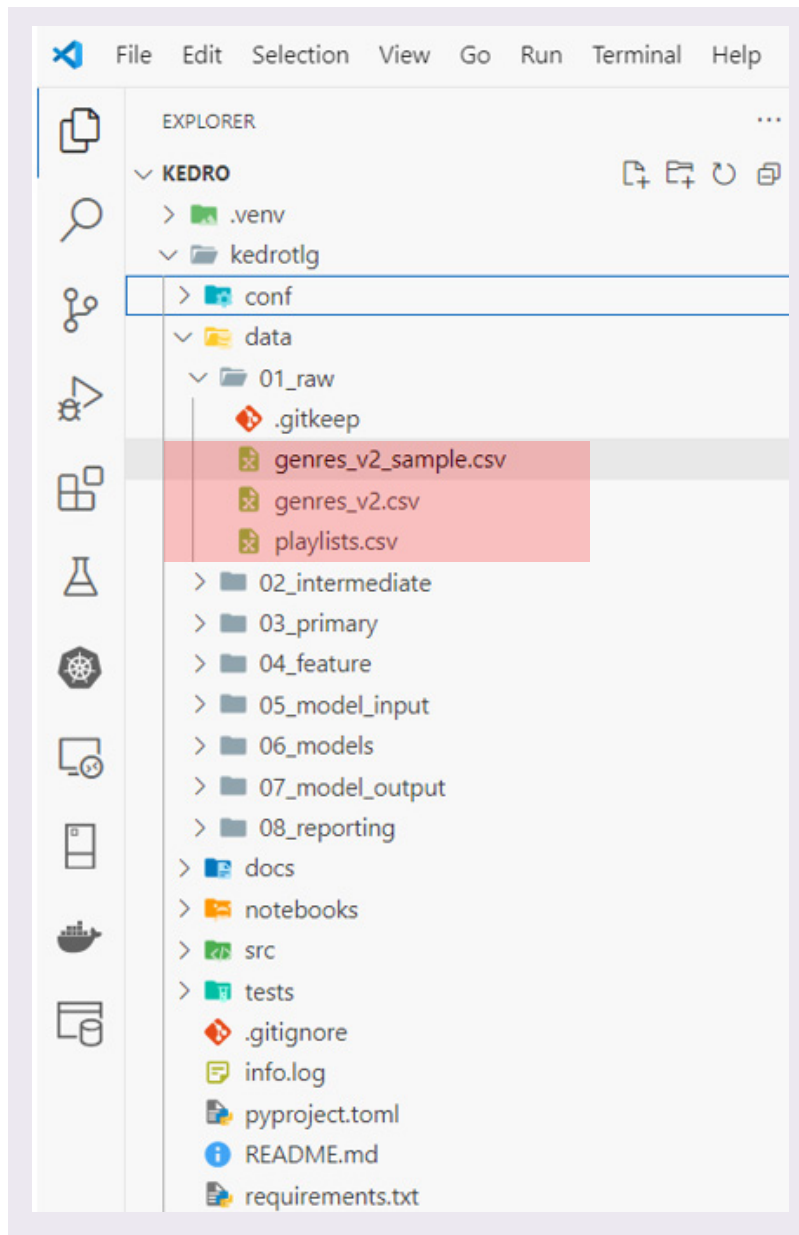


Figura 6. Pantalla de carpeta `data/01_raw`.

Para configurar el *Data Catalog* en un proyecto Kedro, simplemente crea o modifica el archivo `conf/base/catalog.yml` dentro de la carpeta del proyecto.

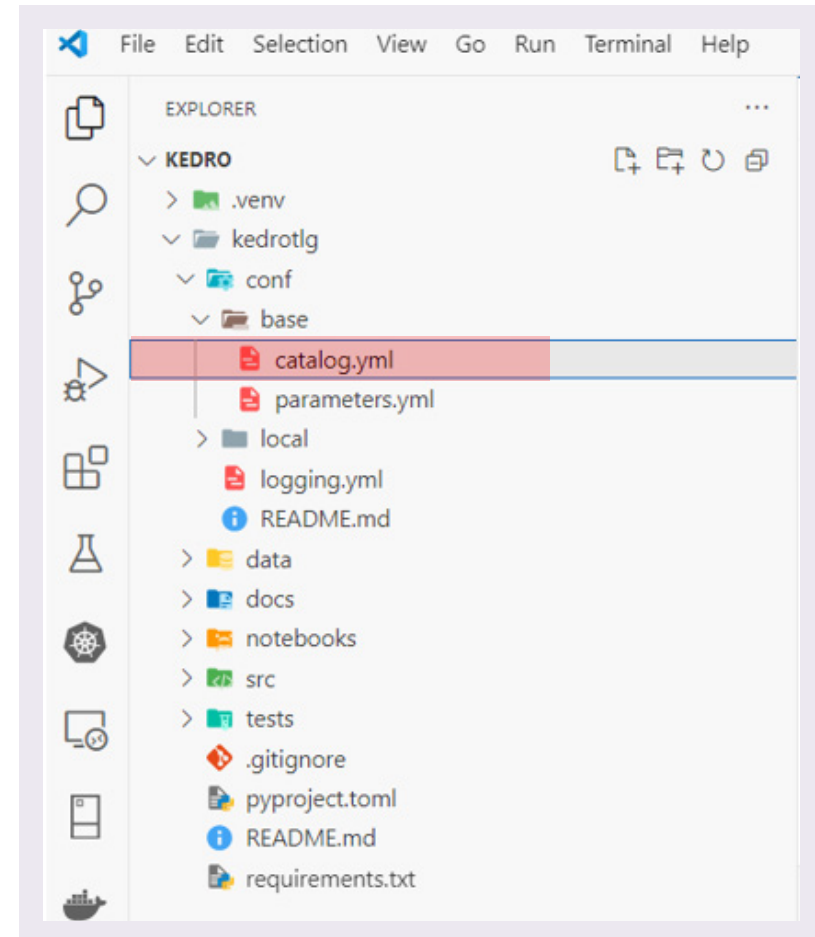


Figura 7. Pantalla para configurar *Data Catalog*.

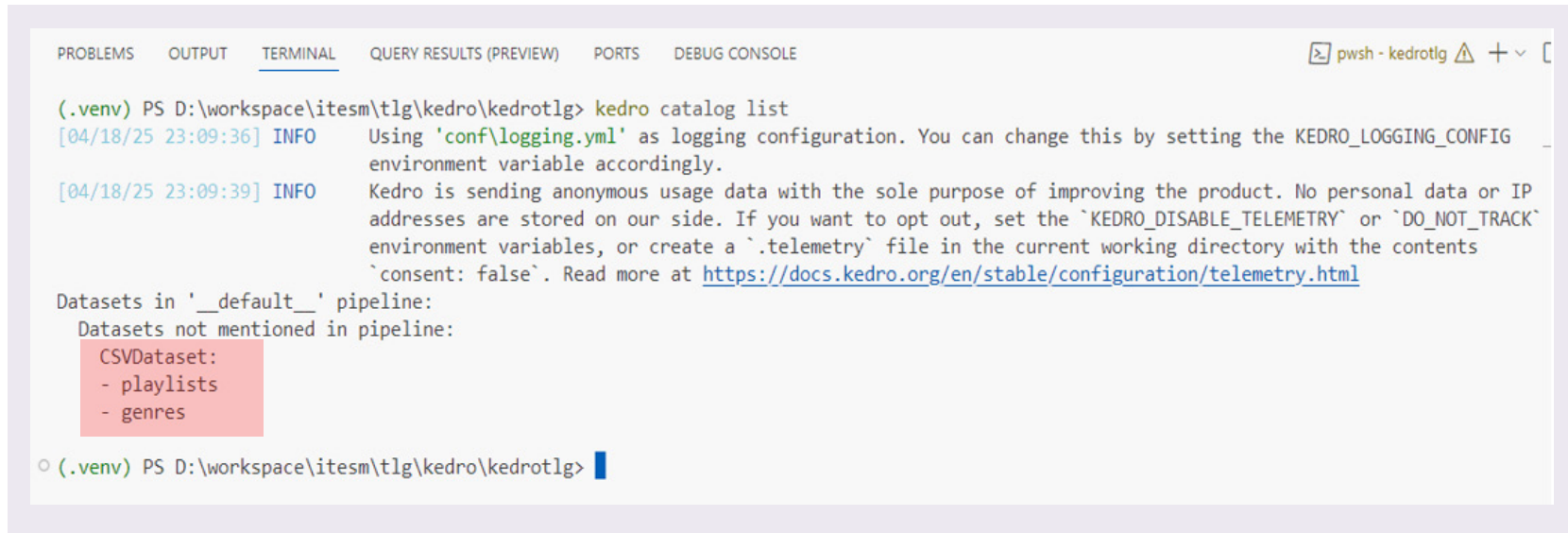
Para configurar los archivos en el catálogo de Kedro, utiliza el siguiente código.

```
genres:  
  type: pandas.CSVDataset  
  filepath: data/01_raw/genres_v2.csv  
playlists:  
  type: pandas.CSVDataset  
  filepath: data/01_raw/playlists.csv
```

Como ves, en este ejemplo se han definido dos *datasets*:

1. **genres**: un archivo CSV que se leerá/escribirá usando pandas.
2. **playlists**: un archivo CSV que también se gestionará con pandas.

Toma en cuenta que podemos revisar si la configuración es correcta ejecutando en una terminal el comando `kedro catalog list` dentro de la carpeta del proyecto, obteniendo el siguiente resultado:



```
PROBLEMS OUTPUT TERMINAL QUERY RESULTS (PREVIEW) PORTS DEBUG CONSOLE
[04/18/25 23:09:36] INFO Using 'conf\logging.yml' as logging configuration. You can change this by setting the KEDRO_LOGGING_CONFIG environment variable accordingly.
[04/18/25 23:09:39] INFO Kedro is sending anonymous usage data with the sole purpose of improving the product. No personal data or IP addresses are stored on our side. If you want to opt out, set the `KEDRO_DISABLE_TELEMETRY` or `DO_NOT_TRACK` environment variables, or create a `.telemetry` file in the current working directory with the contents `consent: false`. Read more at https://docs.kedro.org/en/stable/configuration/telemetry.html

Datasets in '__default__' pipeline:
Datasets not mentioned in pipeline:
  CSVDataset:
  - playlists
  - genres

(.venv) PS D:\workspace\itesm\tlg\kedro\kedrotlg>
```

Figura 8. Pantalla de resultado del comando `kedro catalog list`.

A partir de **Kedro 0.19.0**, los *datasets* no están incluidos en el paquete principal de Kedro. Para usar los *datasets* debes instalar el paquete `kedro-datasets`: **`pip install kedro-datasets`**. En el proyecto actual, este paquete ya está incluido en el archivo `requirements.txt`, pero siempre hay que confirmar.

Una vez instalado el paquete, podrás aprovechar los distintos tipos de *datasets* que ofrece Kedro para manejar datos de forma estructurada y consistente.

Los *datasets* para Kedro son las herramientas que permiten cargar y escribir datos en fuentes y destinos de múltiples tipos. Leen los datos y los convierten automáticamente al formato establecido por el *dataset* de Kedro. En el proyecto actual, `pandas.CSVDataset` indica que se cargará un archivo CSV y que se convertirá en un **DataFrame** de **Pandas**.

Material adicional

- ▶ Kedro. (s.f.). *Conozca Kedro*. Kedro. Recuperado el 01 de abril de 2025 de: <https://docs.kedro.org/en/stable/index.html>
- ▶ Kedro. (s.f.). *Parameters*. [Parámetros]. Kedro. Recuperado el 01 de abril de 2025 de: <https://docs.kedro.org/en/stable/configuration/parameters.html#how-to-specify-parameters-at-runtime>
- ▶ Python. (s.f.). *venv — Creación de entornos virtuales*. Python. Recuperado el 01 de abril de 2025 de: <https://docs.python.org/es/3.13/library/venv.html>
- ▶ Visual Studio Code. (s.f.). Tutorial: Introducción a Visual Studio Code. Visual Studio Code. Recuperado el 01 de abril de 2025 de: <https://code.visualstudio.com/docs/getstarted/getting-started>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.

Bibliografía

- ▶ Kedro. (s.f.). *Create a new Kedro project*. [Crea un nuevo Proyecto en Kedro]. Kedro. Recuperado el 01 de abril de 2025 de: https://docs.kedro.org/en/stable/get_started/new_project.html
- ▶ Kedro. (s.f.). *Introduction to the Data Catalog*. [Introducción al Data Catalog]. Kedro. Recuperado el 01 de abril de 2025 de: https://docs.kedro.org/en/stable/data/data_catalog.html
- ▶ PyPI. (s.f.). *Kedro*. Kedro. Recuperado el 01 de abril de 2025 de: <https://pypi.org/project/kedro/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.

¡Muy bien!

—— Continúa con la lección. ——